

Banker's Algorithm

Problem 1: Safe State Check

A system has five processes (T0-T4) and three resources (A, B, C).

- Total Resources
 - A = 10, B = 5, C = 7
- Allocation Matrix

Process	A	B	C
T0	0	1	0
T1	2	0	0
T2	3	0	2
T3	2	1	1
T4	0	0	2

- Maximum Matrix

Process	A	B	C
T0	7	5	3
T1	3	2	2
T2	9	0	2
T3	2	2	2
T4	4	3	3

Calculate the **Available Resources** and determine whether the system is in a **safe state**. If so, provide the **safe sequence**.

Solutions:

- **Available Resources**
 - Current Allocation per Resource

Process	A	B	C
T0	0	1	0
T1	2	0	0
T2	3	0	2
T3	2	1	1
T4	0	0	2
Total	7	2	5

- Available = Total – Allocated

Resources	A	B	C
Total	10	5	7
Allocated	7	2	5
Available	3	3	2

- **Safe State Determination**

- Need Matrix Calculation (Need = Max – Allocation)

Process	A	B	C		Process	A	B	C		Process	A	B	C
T0	7	5	3		T0	0	1	0		T0	7	4	3
T1	3	2	2		T1	2	0	0		T1	1	2	2
T2	9	0	2		T2	3	0	2		T2	6	0	0
T3	2	2	2		T3	2	1	1		T3	0	1	1
T4	4	3	3		T4	0	0	2		T4	4	3	1
Max Matrix					Allocation Matrix					Need Matrix			

- Find Safe Sequence (Safety Algorithm)

Initial **Available**: (3, 3, 2).

We look for a process whose **Need** $\leq$ **Available**.

1. **T0**: Need (7, 4, 3) > Available (3, 3, 2)
Does not satisfy the requirements.
So, **Safe Sequence** = {NULL}.
2. **T1**: Need (1, 2, 2) $\leq$ Available (3, 3, 2)
T1 can complete execution with the available resources.
After the completion of T1,
Available += Allocation of T1.

Resources	A	B	C
Previously Available	3	3	2
Allocation of T1	2	0	0
Updated Available	5	3	2

So, **Safe Sequence** = {T1}.

3. **T2**: Need (6, 0, 0) > Available (5, 3, 2)
Does not satisfy the requirements.
So, **Safe Sequence** = {T1}.
4. **T3**: Need (0, 1, 1) $\leq$ Available (5, 3, 2)
T3 can complete execution with the available resources.
After the completion of T3,
Available += Allocation of T3.

Resources	A	B	C
Previously Available	5	3	2
Allocation of T3	2	1	1
Updated Available	7	4	3

So, **Safe Sequence** = {T1, T3}.

5. **T4:** Need $(4, 3, 1) \leq$ Available $(7, 4, 3)$
 T4 can complete execution with the available resources.
 After the completion of T4,
 Available += Allocation of T4.

Resources	A	B	C
Previously Available	7	4	3
Allocation of T4	0	0	2
Updated Available	7	4	5

So, **Safe Sequence = {T1, T3, T4}.**

6. **T0:** Need $(7, 4, 3) \leq$ Available $(7, 4, 5)$
 T0 can complete execution with the available resources.
 After the completion of T0,
 Available += Allocation of T0.

Resources	A	B	C
Previously Available	7	4	5
Allocation of T0	0	1	0
Updated Available	7	5	5

So, **Safe Sequence = {T1, T3, T4, T0}.**

7. **T2:** Need $(6, 0, 0) \leq$ Available $(7, 5, 5)$
 T2 can complete execution with the available resources.
 After the completion of T2,
 Available += Allocation of T2.

Resources	A	B	C
Previously Available	7	5	5
Allocation of T2	3	0	2
Updated Available	10	5	7

So, **Safe Sequence = {T1, T3, T4, T0, T2}.**

So, the **Safe Sequence: {T1, T3, T4, T0, T2}.**

Problem 2: Request Evaluation

Using the same data from Problem 1:

Suppose Process T1 requests resources: (1, 0, 2).

- Can the request be granted immediately?
- Would the system remain in a safe state if the request is granted?

Solutions:

- Check if the request can be satisfied with the available resources.
 1. Check if **Request** $\leq$ **Need**
Request of T1 (1, 0, 2) $\leq$ Need of T1 (1, 2, 2).
 2. If **1** is satisfied, check if **Request** $\leq$ **Available**
Request of T1 (1, 0, 2) $\leq$ Available of Resources (3, 2, 2).
 3. If **2** is satisfied, the request can be granted.

So, the request can be granted immediately with the available resources. But the safe state must be checked before granting the request to avoid a deadlock.

- Safe State Checking
 - Updated available

Resources	A	B	C
Previously Available	3	2	2
Request from T1	1	0	2
Updated Available	2	2	0

- Updated Allocation for T1

Resources	A	B	C
Previous Allocation	2	0	0
New Resources Allocation for T1	1	0	2
Updated Allocation	3	0	2

- Updated Need for T1

Resources	A	B	C
Previous Need	1	2	2
New Resources	1	0	2

Allocation for T1			
Updated Need	0	2	0

- Now, check the safety state by applying the safety algorithm like problem 1.
 - T0 (Need: 7, 4, 3), Allocation: (0, 1, 0)
 - T1 (Need: 0, 2, 0), Allocation: (3, 0, 2)
 - T2 (Need: 6, 0, 0), Allocation: (3, 0, 2)
 - T3 (Need: 0, 1, 1), Allocation: (2, 1, 1)
 - T4 (Need: 4, 3, 1), Allocation: (0, 0, 2)

Start with **Available = (2, 3, 0)**

- T0 (Need: 7, 4, 3) > Available (2, 3, 0), (×)
updated available = (2, 3, 0), Safe Sequence={NULL}
- T1 (Need: 0, 2, 0) ≤ Available (2, 3, 0), (√)
updated available=(5, 3, 2), Safe Sequence={T1}
- T2 (Need: 6, 0, 0) > Available (5, 3, 2), (×)
updated available=(5, 3, 2), Safe Sequence={T1}
- T3 (Need: 0, 1, 1) ≤ Available (5, 3, 2), (√)
updated available=(7, 4, 3), Safe Sequence={T1, T3}
- T4 (Need: 4, 3, 1) ≤ Available (7, 4, 3), (√)
updated available=(7, 4, 5), Safe Sequence={T1, T3, T4}
- T0 (Need: 7, 4, 3) ≤ Available (7, 4, 5), (√)
updated available=(7, 5, 5), Safe Sequence={T1, T3, T4, T0}
- T2 (Need: 6, 0, 0) ≤ Available (7, 5, 5), (√)
updated available=(10, 5, 7), Safe Sequence={T1, T3, T4, T0, T2}

So, after granting the request of T1, the system will still stay in a safe state. So, the request can be granted safely.

Problem 3: Construct a Deadlock-Free Scenario

Design a scenario with:

- 3 processes, 4 total instances of a single resource type
- Allocation and Max tables such that:
 - The system is currently not in a safe state
 - But no process is deadlocked

Explain how this is possible using the Banker's algorithm logic.

Solutions:

There is only one resource (A), and the total instance of the resource is four. There are three processes.

Let us assume,

Process	A
T0	3
T1	2
T2	2

Max Matrix

Process	A
T0	2
T1	1
T2	1

Allocation Matrix

So,

Total Allocation for resource, A = 4

Available for the resource, A = 0

So, the Need matrix,

Process	A
T0	1
T1	1
T2	1

Need Matrix

Safety Check for the system,

- T0 need 1 > Available (0), (×)
updated available = (0), Safe Sequence={NULL}
- T1 need 1 > Available (0), (×)
updated available = (0), Safe Sequence={NULL}
- T2 need 1 > Available (0), (×)
updated available = (0), Safe Sequence={NULL}

The system is not in a safe state. But not in deadlock, because processes can still finish if resources become available later.